Particle Physics Simulation

Measuring what the memory hierarchy is worth, and proving the physics is right

Olajide Badejo

November 2026

Abstract

This report describes a CUDA particle physics engine that solves two problems: direct all pairs gravitational N-body dynamics, and weakly compressible smoothed particle hydrodynamics with dense grid neighbour search. It is built around two commitments. Every optimisation is a measured result rather than a claim, and physical correctness is enforced by conservation tests rather than by how the output looks.

The N-body force kernel exists in three variants that isolate one technique each. On an RTX 5070 at 200,000 particles, shared memory tiling takes the step from 84.50 to 43.21 milliseconds, a factor of 1.96, and computes 926 billion interactions per second, 18.52 TFLOP/s at the conventional 20 operations per interaction. Fusing the force evaluation with the integrator was expected beforehand to be worth ten to twenty percent. Round by round it is measurably faster by about one percent over most of the range, and not resolved at the largest size, where tiled trials are bimodal for a reason not yet understood. Hardware counters put device memory throughput below one percent of peak for all three variants, so neither technique works by relieving device memory.

All 14 physics gates pass. A two body circular orbit in single precision holds energy to 1.33×10^{-5} over ten thousand velocity Verlet steps, a figure dominated by rounding, and the integrator's order is confirmed separately in double precision; total momentum in a ten thousand particle Plummer sphere holds to 1.23×10^{-8}; the sampled radial density of that sphere matches the analytic Plummer profile at a chi squared p value of 0.4416. The fluid solver runs 100,000 particles at 0.187 milliseconds per step. This version corrects claims `v1.0.0` got wrong, including a density gate that measured mismatched particles; Chapter 8 lists them.

Contents

1 Introduction **3**
- 1.1 What this is . . . 3
- 1.2 The two commitments . . . 3
- 1.3 The reference machine . . . 4
- 1.4 How to read this report . . . 4

2 Background **5**
- 2.1 The GPU memory hierarchy . . . 5
- 2.2 Gravitational N-body dynamics . . . 5
  - 2.2.1 Why velocity Verlet . . . 6
  - 2.2.2 The Plummer sphere . . . 6
- 2.3 Smoothed particle hydrodynamics . . . 7
  - 2.3.1 Finding neighbours . . . 7

3 Methodology **8**
- 3.1 The kernel ladder . . . 8
- 3.2 Timing protocol . . . 8
- 3.3 Validation design . . . 9
- 3.4 Reproducibility . . . 10

4 Implementation **11**
- 4.1 Layout and state . . . 11
- 4.2 Separating the physics from the kernels . . . 11
- 4.3 The three force kernels . . . 11
- 4.4 The fluid step . . . 11
- 4.5 Input safety . . . 12
- 4.6 Diagnostics without perturbing the run . . . 12
- 4.7 Snapshots and resume . . . 13
- 4.8 The style gate . . . 13

5 Results **14**
- 5.1 The kernel ladder . . . 14
- 5.2 Fusion, expectation against measurement . . . 17
- 5.3 What tiling actually fixes . . . 18
- 5.4 Tile width . . . 19
- 5.5 The fluid solver . . . 21
- 5.6 Physics gates . . . 21

6 Discussion **24**
6.1 Fusion and what the bytes could not say 24
6.2 Tiling is not a bandwidth optimisation here 24
6.3 Where the first design and the measurement disagreed 25
6.4 Limitations 25

7 Conclusion **26**
7.1 Further work 26

8 Corrections to v1.0.0 **27**
8.1 Interior density 27
8.2 Fusion 27
8.3 Tile width 27
8.4 Neighbour search 28
8.5 Provenance 28
8.6 Energy gate 28
8.7 Numbers and prose that had drifted 28

Chapter 1

Introduction

1.1 What this is

This is a particle physics engine written in CUDA. It solves two problems that sit at opposite ends of the memory access spectrum, which is the reason both are here.

The first is direct all pairs gravitational N-body dynamics. Every particle attracts every other, so the force evaluation is $O(N^2)$ arithmetic over $O(N)$ data. Access is perfectly regular and entirely predictable.

The second is smoothed particle hydrodynamics, in which a particle interacts only with the neighbours inside a finite smoothing radius. The work is nearly linear in the particle count, but which particles are neighbours changes every step and has to be discovered before any physics can be evaluated. Access is irregular and data dependent.

A GPU rewards the first pattern and punishes the second, and the techniques that matter are different in each case. Putting both in one engine, behind one configuration format and one integrator, makes the contrast concrete.

1.2 The two commitments

No number without a run. Every performance figure in this report was produced by a program in this repository, recorded to a comma separated file with the commit that produced it, and read back into this document mechanically. The only typed values are `v1.0.0` values in correction records and a few figures from one off runs that no file records, such as the double precision truncation error in Section 5.6. Where I had an expectation before measuring, the expectation is stated and then compared with what actually happened. Where a published claim turned out to be wrong, Chapter 8 says so.

Correctness is enforced, not observed. A particle simulation that is subtly wrong still looks like a particle simulation. Fluid still sloshes and stars still orbit when the force law has a sign error in one term or the neighbour search quietly counts some particles twice. The defence is a set of conservation and distribution tests with numerical thresholds, run by the test suite on a machine with a GPU, that fail loudly. There are 14 of them and Chapter 5 reports what each one measured.

These two commitments interact. The benchmark builds use `fast math`, which lowers the reciprocal square root to a hardware approximation. The physics gates should not depend on a flag like that. So the build compiles the device library twice from identical sources, once with fast math for the benchmarks and once without for the gates. A gate that a compiler flag can relax is not a gate.

1.3 The reference machine

Everything here was measured on one machine: an NVIDIA GeForce RTX 5070 of compute capability 12.0 with 48 streaming multiprocessors on a 192 bit memory bus, driven from Ubuntu under the Windows Subsystem for Linux with CUDA 13.3. Its theoretical peak memory bandwidth is 672.0 GB/s. A streaming copy kernel achieves 575.1 GB/s, or 85.6 percent of that peak, and it is the measured figure rather than the theoretical one that every bandwidth statement in this report is compared against. Quoting utilisation against a peak no kernel can reach flatters every kernel by the same factor while saying nothing about any of them.

1.4 How to read this report

Chapter 2 covers the GPU memory hierarchy and the physics of both solvers, including why velocity Verlet rather than anything else. Chapter 3 describes the kernel ladder, the timing protocol and the design of the validation suite. Chapter 4 describes what was built. Chapter 5 is the measurements. Chapter 6 interprets them, including the results that contradicted my expectations. Chapter 8 lists what `v1.0.0` got wrong. A companion document, `debug_report.pdf`, records the defects found along the way and how each was diagnosed.

Chapter 2

Background

2.1 The GPU memory hierarchy

The arithmetic units of a modern GPU are far faster than the path that feeds them. The reference card sustains 18.52 TFLOP/s of single precision arithmetic on this workload, by the convention Chapter 5 states, while its device memory delivers 575.1 GB/s in practice. At four bytes per float that is about 144 billion floats per second, so a kernel that performs one operation per float loaded runs at under one percent of the arithmetic the hardware has shown it can do. Every technique in this project is a way of raising the number of operations performed per byte brought in from device memory.

Three levels of the hierarchy matter here.

Coalescing. A warp of thirty two threads issues its loads together. When those threads read consecutive addresses, the hardware merges them into a small number of wide transactions. When they read scattered addresses it issues many narrow ones and discards most of what it fetched. This is why the particle state is stored as a structure of arrays rather than an array of structures: if one particle's seven floats sat together, a warp reading the x component of thirty two consecutive particles would touch thirty two separate cache lines and use a fraction of each. With separate arrays the same read is thirty two consecutive floats [7].

Shared memory. Each multiprocessor has a small block of memory, 48 KiB per block on this card, that all threads of a block can read at a latency and bandwidth far better than device memory. A block can cooperatively stage a tile of data there once and then have all its threads read it many times. This is the mechanism behind the tiled force kernel.

Registers and the load store path. Values held in registers cost nothing to read. Between registers and device memory sit the first level cache and the load store pipeline, and as Chapter 5 shows, on this problem that pipeline rather than device memory is what actually limits the naive kernel.

2.2 Gravitational N-body dynamics

The acceleration of body i under the gravity of all the others is

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\left(|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2 \right)^{3/2}}. \tag{2.1}$$

The softening length ε in the denominator is not a numerical convenience to be apologised for. Without it the acceleration diverges as two bodies approach, a single time step throws the

pair apart at enormous speed, and the run is destroyed. For a collisionless system, where the smooth mean field rather than individual close encounters carries the dynamics, softening is the physically appropriate model [2]. The shipped runs are not fully in that regime, as Chapter 6 notes, and there softening is what keeps the integrator stable. The potential must be softened consistently,

$$U = -G \sum_{i<j} \frac{m_i m_j}{\sqrt{|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2}}, \tag{2.2}$$

because pairing a softened force with an unsoftened potential breaks energy conservation for a reason that has nothing to do with the integrator. Both come from the same expression in the code, and a test differentiates Equation 2.2 numerically and compares it against Equation 2.1.

2.2.1 Why velocity Verlet

The integrator advances

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t + \tfrac{1}{2}\mathbf{a}_n \Delta t^2, \tag{2.3}$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \tfrac{1}{2}\left(\mathbf{a}_n + \mathbf{a}_{n+1}\right)\Delta t. \tag{2.4}$$

Three alternatives were considered and rejected.

Explicit Euler is first order and not symplectic. Its energy error grows without bound, and on a circular orbit it spirals outward.

Fourth order Runge Kutta is accurate per step but requires four force evaluations per step. Since the force evaluation is essentially the entire cost of this simulation, that is a factor of four in exchange for accuracy that a symplectic method achieves differently.

Velocity Verlet is second order, needs exactly one force evaluation per step, and is symplectic: it exactly conserves a modified Hamiltonian near the true one, so its energy error oscillates within a bound rather than accumulating [11, 4]. The single precision energy figure in Section 5.6 is too dominated by rounding to show this, so the order is tested in double precision.

The fused kernel uses the leapfrog form,

$$\mathbf{v}_{n+1/2} = \mathbf{v}_{n-1/2} + \mathbf{a}_n \Delta t, \tag{2.5}$$

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1/2} \Delta t, \tag{2.6}$$

which produces the same position sequence to rounding, provided the velocity is offset backwards by half a step once before the loop begins. That offset, and the fact that the stored velocity afterwards is not synchronised with the positions, is a detail with real consequences for snapshots and for the energy diagnostic; both are handled explicitly in Chapter 4.

2.2.2 The Plummer sphere

Benchmarks need a physically sensible initial condition. A uniform random cloud is not one: it is far from equilibrium and collapses immediately, so it measures the wrong thing. The Plummer sphere [9] has density

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}, \qquad \frac{M(r)}{M} = \frac{r^3}{(r^2 + a^2)^{3/2}}, \tag{2.7}$$

and the cumulative profile inverts in closed form, so radii are sampled directly. Speeds are drawn from the exact distribution function by the rejection method of Aarseth, Hénon

and Wielen [3], which puts the sphere in virial equilibrium at $t = 0$. Drawing speeds from a convenient Maxwellian instead would leave it out of equilibrium, and it would visibly breathe on the first crossing time in a way that looks exactly like an integrator bug.

2.3 Smoothed particle hydrodynamics

SPH represents a continuous field by interpolating over particles with a smoothing kernel W of finite support h [5]. Density is

$$\rho_i = \sum_j m_j W(|\mathbf{r}_i - \mathbf{r}_j|, h), \tag{2.8}$$

and pressure follows from a stiff equation of state $p = k(\rho - \rho_0)$, which keeps the fluid weakly compressible without solving a pressure Poisson equation every step.

Three different kernels are used, each for the one job it does well [6]. The *poly6* kernel is a polynomial in r^2, so density needs no square root. The *spiky* kernel supplies the pressure gradient, because its gradient does not vanish as the separation goes to zero; the poly6 gradient does, which means no restoring force exactly where the fluid needs one most, and particles pile up on top of each other. The *viscosity* kernel has a positive Laplacian throughout its support, so the viscous term always damps and never injects energy. Each integrates to one over its support, which is the property the normalisation gate checks by quadrature.

2.3.1 Finding neighbours

The neighbour set changes every step and must be rebuilt every step. Every fluid domain here is bounded and has walls, so the search uses a dense uniform grid, with cells at least as wide as the smoothing radius, and a neighbour can then only be in one of the twenty seven cells around a particle. Cell (i, j, k) of an $n_x \times n_y \times n_z$ grid has the index

$$c(i, j, k) = i + n_x (j + n_y k), \tag{2.9}$$

and particles are sorted by it. Two things follow. The index is unique, so two visited cells can never be the same slot, and no neighbour can be counted twice. And x varies fastest, so the three cells of an x row are contiguous in sorted order, and the twenty seven cells are nine ranges. Section 5.5 reports the measured agreement with a brute force sum, which is what proves it.

Earlier versions used spatial hashing [10] with a table of twice as many slots as particles, on the argument that a dense grid would waste its storage on empty cells. That argument was wrong, as Chapter 8 records: on these domains the table was larger than the grid it avoided. A k-d tree is a poor fit for a full rebuild on a GPU every step.

Chapter 3

Methodology

3.1 The kernel ladder

The point of the ladder is attribution. Three force kernels differ by exactly one technique each, so the difference between two rungs can be assigned to that technique and nothing else. All three compute the same quantity from the same structure of arrays layout, and a test asserts they agree with a double precision host reference to better than one part in ten thousand, so a performance difference cannot be a difference in what is being computed.

Rung one, naive. Every thread walks the whole particle array in device memory. This is not a straw man built to lose. All threads in a warp read the same index on the same cycle, so the load is a broadcast and the line stays resident for the whole warp. What it does not do is share that line between the warps of a block, so the same data crosses the cache boundary once per resident block rather than once per tile.

Rung two, tiled. The block cooperatively stages a tile of source particles in shared memory and every thread of the block consumes it, following Nyland, Harris and Prins [8]. The default tile width is 256. The cost is two barriers per tile, so narrow tiles pay the barrier more often per unit of arithmetic; Section 5.4 measures that trade rather than arguing about it.

Rung three, fused. The tiled force evaluation and the integrator update happen in one launch instead of three, which brings the leapfrog form of the integrator with it. Both of the split path's integrator launches read the acceleration array, $24N$ bytes per step, and the fused launch reads it in neither. The acceleration is still written, $12N$ bytes, although the engine no longer reads that write: the diagnostics and the velocity conversions recompute the force first. Dropping the write is a kernel change and is left for a later baseline.

Fusion has a correctness constraint that is worth stating because it is easy to get wrong. There is no grid wide barrier inside a launch. If the kernel updated positions in place, a block that had finished its force sum could overwrite a position that another block was still reading, and the result would depend on scheduling. Positions are therefore double buffered: read from one array, written to the other, swapped afterwards. This costs $12N$ bytes of memory and no bandwidth at all, because the write has to happen either way.

3.2 Timing protocol

Three things ruin a GPU timing measurement, and the protocol addresses each.

The first launch of a kernel pays for module loading and for the card ramping up its clocks, so every measurement is preceded by warm up steps that are not timed.

The host can queue work far faster than the device retires it, so timing on the host measures the host. All timings use CUDA events recorded on the same stream as the work.

Run length varies enormously across the sweep, from a fraction of a millisecond to nearly a hundred. Each point is therefore calibrated first with a single timed step, and the run length is chosen to make each trial last about 120 ms, between 20 and 400 steps.

A fourth thing ruins a comparison, which is session drift. More trials do not remove it, because drift is confounded with which variant ran when. So the configurations being compared are measured in rounds: every configuration's first trial, then every configuration's second trial, and so on for ten rounds, with the order inside a round rotating and each engine staying alive between its trials. A group is the three variants at one particle count, the fluid sizes, or the tile widths.

Each row reports the median of its ten trials, which one interrupted trial cannot move, with the minimum, mean, standard deviation and interquartile range beside it. Comparisons between variants come from the raw trials: the time ratio inside each round, then the median and interquartile range of those ratios, because two configurations in the same round saw the same machine. The sweep also measures one reference configuration at its start, middle and end, here tiled at 100,000 particles, and the spread of those three medians, 0.19 percent, is the session drift. A difference smaller than that is not quoted as a difference.

`v1.0.0` did something else: it measured each variant in its own block, kept the fastest of three trials, and asserted that relative differences were stable to a tenth of a percentage point. Its own session spread was about 3 percent, which is why its smallest differences have been withdrawn in Chapter 8. Timings from the two protocols are not directly comparable.

3.3 Validation design

Each gate is chosen to fail for a specific class of defect rather than to produce a reassuring number.

Two body energy error. A circular orbit integrated for ten thousand steps in single precision must hold total energy to better than one part in a thousand. This catches a broken force or update path, an inconsistent softened potential and a mis-derived initial orbital speed. The initial speed is computed from the softened force law, so the orbit is genuinely circular rather than a slowly precessing ellipse. It cannot see the integrator's order, because its value is almost entirely rounding.

Integrator order. Double precision runs at several time steps measure the slope of the energy error, which must come within 0.15 of two on an orbit of eccentricity 0.5, the method's order, and within 0.15 of four on an unsoftened circular orbit, where symmetry cancels the second order term.

Momentum conservation. The internal forces of an isolated system cancel exactly in the mathematics. On the device they cancel only to rounding, because each thread accumulates its own sum in single precision in its own order. The check bounds the residual against what single precision can explain, $\epsilon_{\mathrm{f32}}\sqrt{N}n_{\mathrm{steps}}$, rather than against an arbitrary constant. That bound is six orders of magnitude above the measured value, so this is a sanity check.

Force antisymmetry. Newton's third law checked pair by pair against an independent double precision host implementation, written as literally as the force law can be written, with no tiling and no reciprocal square root. Its only job is to be obviously correct.

Plummer radial density. A chi squared test of the sampled radial distribution against the analytic cumulative mass profile. Shells are chosen to hold equal expected counts, so no bin is too sparse for the statistic to be valid. A sampler with a subtly wrong exponent produces a blob that looks perfectly reasonable and fails this immediately. The test is itself checked by running it against a deliberately wrong scale radius, which it must reject.

SPH kernel normalisation. Each smoothing kernel is integrated over its support by composite Simpson quadrature and must come to one within 10^{-3}. A kernel whose normalisation is off by a few percent produces a density field wrong by the same few percent everywhere, and the fluid still looks like a fluid.

Neighbour search against brute force. The density field from the neighbour search is compared against an $O(N^2)$ host sum over the same positions. This is what would catch a dropped or double counted neighbour, and it is the direct test of the grid described in Chapter 2.

Interior density. A jittered block at the calibrated mass is measured away from its free surface, with positions and densities read in the same order. The mean error must stay below 1 percent, the RMS deviation below 5 percent and the maximum deviation below 10 percent. The calibration sets the mean, so that limit is a consistency check; the RMS deviation is driven by the jitter and is the real gate. These limits were committed before the code that measures them.

Recording. An ordinary test run writes its values to the build tree and leaves the committed results alone. Only `make record-results` updates them, and a script fails if a registered gate has no row.

3.4 Reproducibility

Every configuration carries an explicit seed and the initial conditions are generated on the host from a written out uniform mapping rather than from a standard library distribution, whose sequence is not specified across implementations. Each benchmark row records the seed, the commit and whether the tree was dirty, the protocol, the CUDA runtime and driver versions, the fast math setting and a timestamp. `bench_all` refuses to record from a dirty tree, and `scripts/check_provenance.py` fails if a committed row's commit does not resolve. Snapshots are written to a temporary file and renamed into place, so an interrupted run never leaves a half written snapshot for a later resume to pick up, and a test asserts that stopping and resuming reproduces a straight through run bit for bit.

Chapter 4

Implementation

4.1 Layout and state

Particle state lives on the device for the whole run as thirteen float arrays: three positions, three staged positions for the fused path, three velocities, three accelerations and the mass. At two hundred thousand particles that is about ten megabytes, so memory is never a constraint at these sizes and the layout decision is entirely about access pattern.

The host holds no particle data during a run. Configuration and diagnostic accumulators are the only host allocations, and the snapshot staging buffer is touched only when a snapshot is actually written.

4.2 Separating the physics from the kernels

The force law, the smoothing kernels, the equation of state and the uniform grid all live in headers marked so they compile both as device code and as ordinary host C++. This buys two things. The kernels and the host side tests cannot drift apart, because they are the same code. And the formula level tests, which are the majority of them, run in continuous integration on a machine with no GPU.

4.3 The three force kernels

The tiled kernel has one subtlety worth recording. Threads whose index is past the end of the array must not return early, because they still have to reach both barriers on every iteration of the tile loop; a barrier that only some threads of a block arrive at is undefined behaviour. They run the whole loop with clamped reads and discard the result at the end. Padded tile slots carry zero mass, so they contribute nothing and the inner loop needs no bounds test at all. Self interaction is likewise harmless and is deliberately not special-cased: the numerator vanishes while the softened denominator stays finite, so the diagonal term contributes exactly zero and the inner loop stays branch free.

4.4 The fluid step

One fluid step is six device passes: cell index, sort, cell bounds, gather, density, and a combined force and integration pass.

The sort is a CUB radix sort of cell and particle index pairs [1]. Only the bits that can differ are sorted: the number of cells bounds the key, so the sort stops at the significant bit count rather than walking all thirty two bits. A non finite position has no cell and takes the

key one past the last, which the bit count includes. Every finite particle sorts ahead of it, and no search reads it.

The cell bounds pass fills a table with the number of particles before each cell, so a row of cells is two reads and an empty cell is an empty range. A cell is slightly wider than the smoothing radius, so that rounding cannot put two neighbours two cells apart.

After sorting, particle data is gathered into cell sorted order, and the permutation is kept. The particle arrays end a step in the order the next step wants, so the sort has an effect that outlives the step that paid for it. Because that permutation must be a bijection, one gate checks that the multiset of particle masses is unchanged across a step and another checks that total mass is conserved over two thousand steps; a gather that lost or duplicated an index would show up in both.

The force and integration pass is fused for the same reason the gravitational one is, and it is safe for the same reason: the sorted arrays it reads are a separate copy from the primary arrays it writes.

The update is $v \leftarrow v + a\,\Delta t$, then $x \leftarrow x + v\,\Delta t$, the same kick drift leapfrog as the fused gravitational kernel. The initial half step offset is not applied, because the acceleration is still zero when the engine would apply it, so the initial velocity is taken as $v_{-1/2}$ as it stands. The viscous term reads the half step velocity, which makes it first order.

4.5 Input safety

A misspelled or out of range configuration key aborts with a message naming it, and zero softening is refused for N-body, because it makes every run produce not a number. The fluid solver computes acoustic, viscous and body force limits on the time step and refuses a step above the smallest, unless `--allow-unstable-dt` turns the refusal into a warning. Non finite state stops a run, and a snapshot of it is not written.

4.6 Diagnostics without perturbing the run

The particle state is single precision, but the reduction that measures conservation is not. A float32 tree reduction over ten thousand terms loses three to four decimal digits, and the energy gate resolves one part in a thousand over ten thousand steps; reducing in single precision would have measured the reduction rather than the integrator. Accumulators are double, and the block partials are summed in a fixed order on the host rather than with atomics, so two runs of the same configuration agree bit for bit.

The fused variant carries the leapfrog half step velocity, which is not the velocity belonging to the current positions. Recovering the synchronised velocity by shifting the array forward and then back would not return the original bits, and that would break the bit identity guarantee for any run that happened to take a diagnostic. The correction is applied inside the reduction kernel instead, and a test asserts that taking a diagnostic on every step leaves the trajectory unchanged. On the fused path the engine recomputes the force before each reduction, because the kernel leaves the previous acceleration in the array and the synchronised velocity needs the next one. The fused kernel never reads that array, so the extra pass cannot affect the trajectory.

Each diagnostic row costs an all pairs potential pass and two synchronous copies, so the default interval is one hundred steps. A resumed run appends to its diagnostics file rather than truncating it. Solver time, from device events around the stretches of steps between diagnostics and snapshots, and wall time are reported separately, and neither counts the steps before a resume.

4.7 Snapshots and resume

The snapshot format is `CPS2`: a 64 byte header, a table with one 40 byte entry per array, and the payload the table points into. The header holds a 64 bit step, the simulated time as a double, the particle count, a fingerprint of the physics configuration and a checksum of the payload. A reader skips array names it does not know, so a later version can add arrays without another format change. A resume is refused if the checksum fails or the fingerprint differs, and `--force-resume` overrides the fingerprint. `CPS1`, the sixteen byte header of `v1.0.0`, is still read with a warning and is never written.

The flags word carries the one thing a reader cannot infer from the numbers, namely whether the stored velocities are synchronised with the positions or offset by half a step. Resuming a fused run under the wrong convention would shift the whole trajectory by half a step, which looks exactly like a subtly wrong force.

Accelerations are not stored. They are a deterministic function of the positions, so recomputing them on load is cheaper than storing them and guarantees the resumed run enters the loop in exactly the state the original run held. The fluid solver keeps no force pass outside its step, so after a fluid resume the first diagnostic row reads a zero acceleration; the trajectory is unaffected and stays bit identical.

4.8 The style gate

The repository forbids en dashes and em dashes. Enforcing that on source files is easy; enforcing it on the reports is not, because TeX silently turns two hyphens into an en dash. The gate therefore also parses the LaTeX sources for that ligature, excluding verbatim regions where a command line flag legitimately contains two hyphens, and runs over the compiled PDFs through a text extractor. It caught a real instance during this build: the page ranges in the bibliography, written in the conventional BibTeX style, would have rendered as en dashes in the reference list of the finished document.

Chapter 5

Results

Every number in this chapter was read mechanically out of a file written by a run at commit `6823f76a56af`, on the machine described in Chapter 2, with fast math on for the benchmark library and off for everything in Section 5.6. Session drift over the sweep was 0.19 percent, and a difference smaller than that is not quoted as a difference.

5.1 The kernel ladder

	milliseconds per step			speedup		tiled
particles	naive	tiled	fused	tiled	fused	TFLOP/s
10,000	0.467	0.154	0.148	3.04	3.16	13.03
20,000	1.066	0.535	0.527	1.99	2.02	14.95
50,000	6.638	3.211	3.199	2.07	2.07	15.57
100,000	22.851	11.488	11.384	1.99	2.01	17.41
150,000	48.797	24.783	24.568	1.97	1.99	18.16
200,000	84.499	43.208	43.206	1.96	1.96	18.52

Table 5.1: The N-body ladder at the default tile width of 256. Median milliseconds per step for each variant, the speedup of each against the naive baseline, and the throughput of the tiled kernel at the conventional 20 operations per interaction.

	naive to tiled		tiled to fused	
particles	time saved	speedup	time saved	speedup
10,000	+67.1%	3.043	+3.8%	1.040
20,000	+49.8%	1.992	+1.5%	1.015
50,000	+51.6%	2.067	+0.4%	1.004
100,000	+49.7%	1.989	+0.9%	1.009
150,000	+49.2%	1.969	+0.9%	1.009
200,000	+48.9%	1.956	+0.0%	1.000

Table 5.2: What each rung of the ladder is worth on its own, from the medians of Table 5.1. The first pair of columns is the effect of shared memory tiling; the second is the effect of fusing the integrator into the force kernel. Table 5.3 is the better evidence for the second pair.

Tiling is worth a factor of 1.96 at 200,000 particles, taking the step from 84.50 to 43.21 milliseconds. The paired ratio of tiled to naive there is 0.5112, with an interquartile range of

	fused / tiled		tiled / naive	
particles	median	IQR	median	IQR
10,000	0.9624	0.0043	0.3289	0.0016
20,000	0.9848	0.0017	0.5021	0.0010
50,000	0.9922	0.0065	0.4842	0.0033
100,000	0.9919	0.0032	0.5028	0.0014
150,000	0.9905	0.0012	0.5079	0.0006
200,000	0.9913	0.0097	0.5112	0.0039

Table 5.3: Time ratios taken round by round from the raw trials, which cancels session drift. Median and interquartile range over 10 rounds. A ratio below one favours the first variant named.

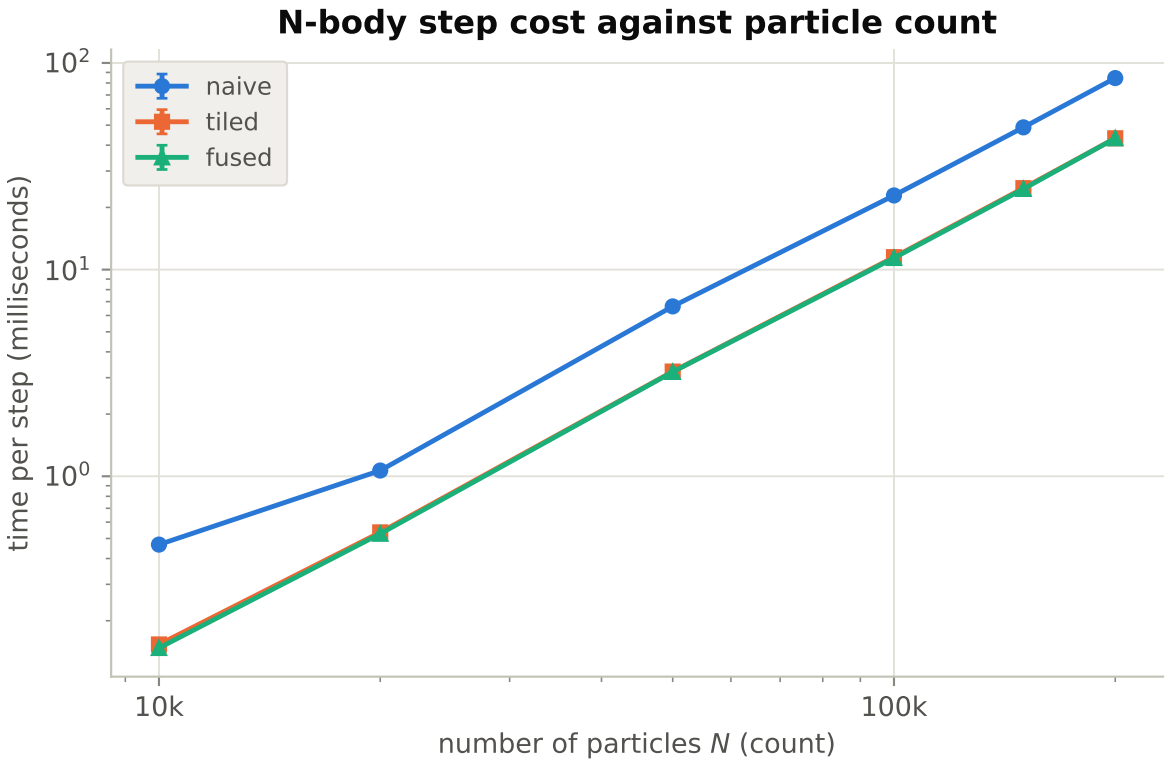


Figure 5.1: Step cost against particle count for the three variants.

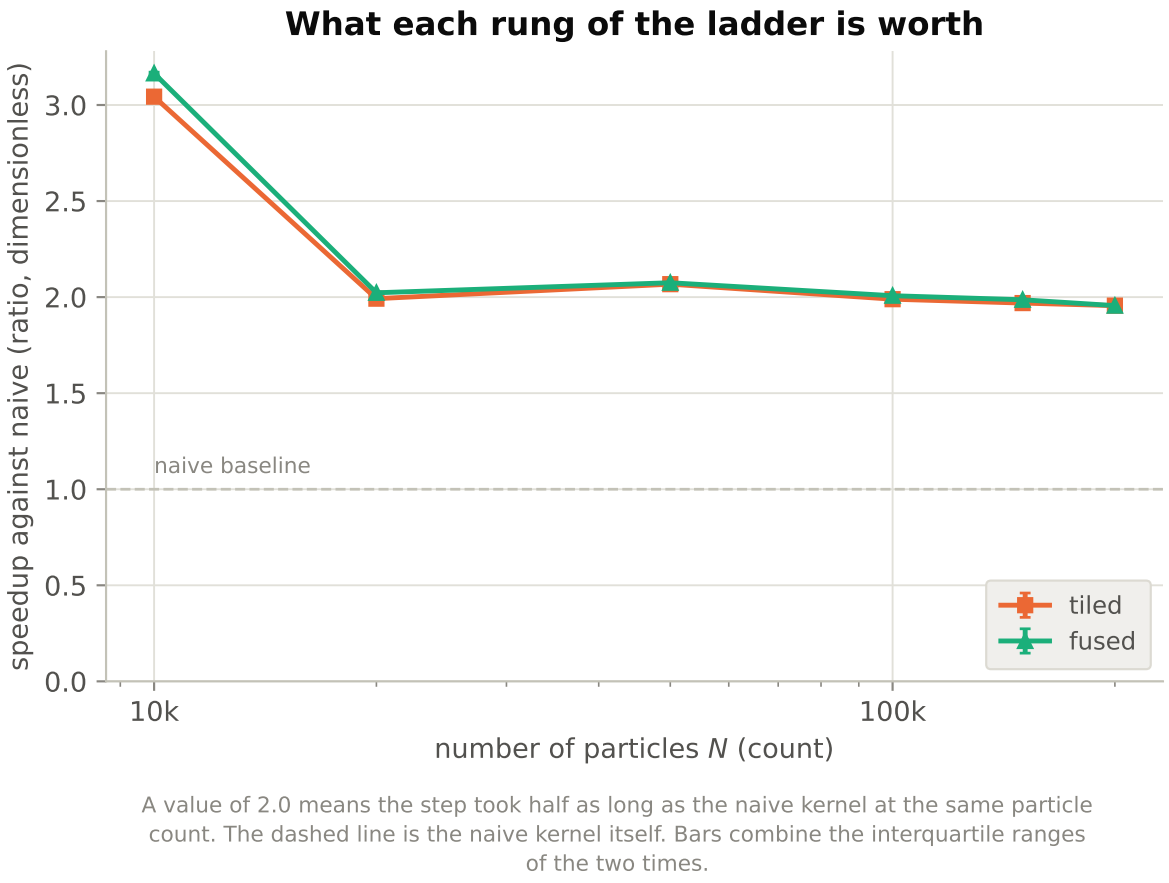


Figure 5.2: The same data as a speedup against the naive baseline. The tiled and fused lines lie close together; Section 5.2 reads the gap between them.

0.0039. The factor does not grow with particle count: it is 3.04 at 10,000 particles and close to two from twenty thousand up, 1.99 there and 1.96 at the largest size. `v1.0.0` said here that the advantage grows with particle count and gave a mechanism for it, and its own table contradicted both. I have no measurement that explains the larger factor at 10,000.

At the largest size the tiled kernel computes 926 billion pairwise interactions per second. That count is the measurement. Turning it into floating point operations takes a convention: at the 20 operations per interaction used by Nyland, Harris and Prins [8] it is 18.52 TFLOP/s, and at the 18.0 the hardware counters report, counting a fused multiply add as two, it is 16.70 TFLOP/s. Every TFLOP/s figure in this report uses the convention, so it can be compared with other work that does. The peak anywhere in the sweep is 18.52 TFLOP/s on the same basis.

5.2 Fusion, expectation against measurement

Before measuring, I expected fusion to be worth ten to twenty percent. It is not worth that at any size. Table 5.3 gives the fused time over the tiled time. At 10,000 particles the ratio is 0.9624, with an interquartile range of 0.0043, and at one hundred and fifty thousand it is 0.9905, with a range of 0.0012. From 10,000 to one hundred and fifty thousand the gain is larger than both its own interquartile range and the session drift of 0.19 percent, so at the default tile width the fused kernel is measurably faster, by about one percent over most of that range.

At 200,000 particles the difference is not resolved. The ratio is 0.9913 with an interquartile range of 0.0097, because the tiled trials there are bimodal: 3 of the 10 rounds sit near 43.6 milliseconds and the rest near 43.2. I do not yet understand why. One hundred and fifty thousand particles shows no such split.

What fusion removes is known from the code. The split path runs three kernels: half kick and drift, force, closing half kick. Both integrator kernels read the acceleration array, $24N$ bytes per step, and the velocities are read and written twice. The fused path runs one kernel and does not read the acceleration array at all. It still writes it, $12N$ bytes, although the engine never reads that write, because the diagnostics and the velocity conversions recompute the force first. Counting each array once per kernel, and leaving out the repeated source reads of the force sum, which both paths share and which dominate, the split path touches 124 bytes per particle per step and the fused path 64, so fusion removes $60N$ bytes and two kernels. The split path in these tables launches as a captured CUDA graph, which removes time between its kernels and so takes away some of what fusion had to offer, while the fused kernel is not captured. With no graphs, the ratio at ten thousand particles was 0.9445.

That traffic does not explain the gain. At the measured streaming bandwidth of 575.1 GB/s, $60N$ bytes take 0.0209 milliseconds at 200,000 particles, which is 0.048 percent of a step, and 0.68 percent at 10,000. The measured gains are several times larger at every resolved size. `v1.0.0` called a figure of this kind the absolute ceiling on what fusion could deliver. It is not a ceiling: it prices one resource and says nothing about the rest of the step. The time saved rises with particle count up to one hundred and fifty thousand, so a fixed launch cost cannot be all of it either. The source of the gain has not been measured.

The hardware counters do show why the expectation of ten to twenty percent was too high.

Device memory throughput is 0.34 percent of peak for the naive kernel and 0.32 percent for the tiled one. The all pairs sum is $O(N^2)$ arithmetic over $O(N)$ data, and device memory is almost idle throughout, so there was no device memory pressure for fusion to relieve.

	throughput, percent of peak				per launch	
variant	SM	L1 and load store	L2	device memory	load sectors	warp issue
naive	87.96	87.96	1.70	0.34	5,000.1M	54.98
tiled	82.84	45.50	1.75	0.32	78.3M	82.84
fused	83.49	45.85	1.71	0.35	78.3M	83.49

Table 5.4: Nsight Compute counters for one launch at 200,000 particles. Device memory throughput is below one percent of peak for every variant.

5.3 What tiling actually fixes

Table 5.4 also corrects a second assumption. Tiling is usually described as a bandwidth optimisation, and on this problem it is not one: device memory throughput is negligible both before and after. What tiling reduces is pressure on the first level cache and the load store pipeline.

Global load sectors fall by a factor of 63.9 and load store instructions by a factor of 3.8. First level cache throughput falls from 88.0 percent of peak, which is close to saturation and the highest figure in the naive row, to 45.5 percent. Warp issue efficiency rises from 55.0 percent to 82.8 percent. The naive kernel's SM throughput equals its first level cache figure, and the tiled kernel's equals its issue figure, which points at the limiting unit of each.

The arithmetic instruction counts are the control in this experiment. They are equal across the three variants to within a fraction of a percent, confirming that the variants differ only in how they move data.

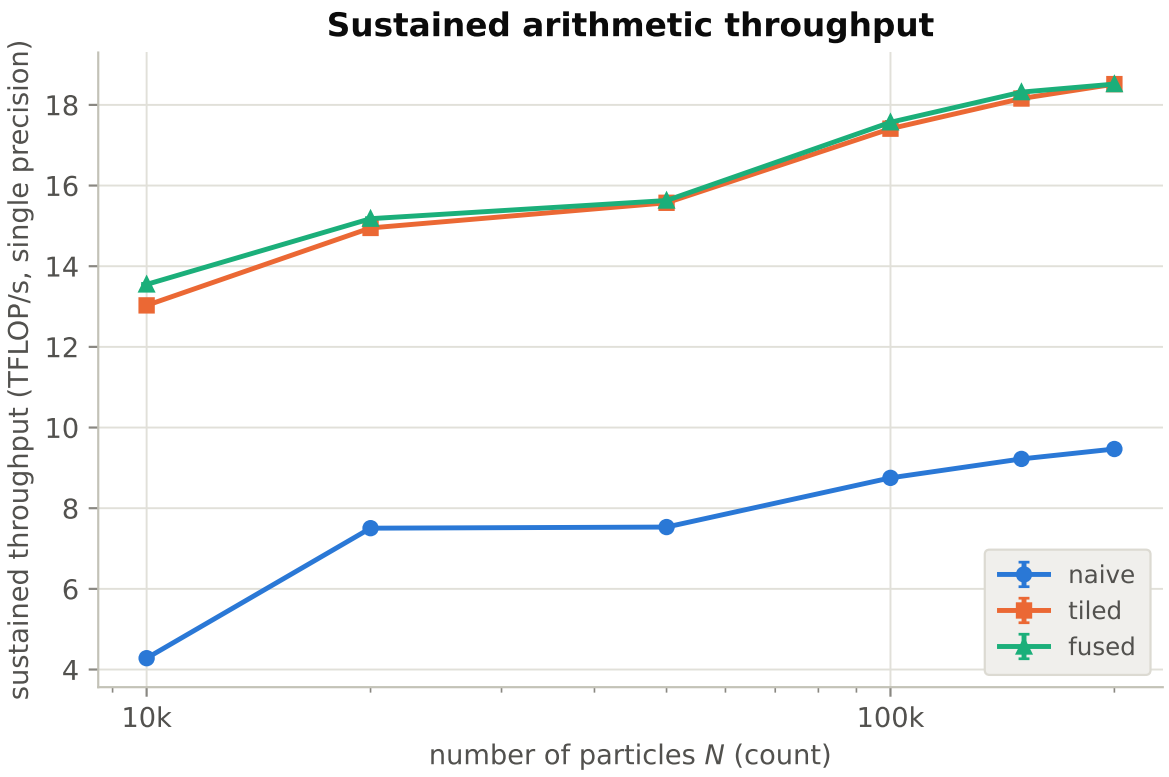


Figure 5.3: Throughput of each variant at the conventional 20 operations per interaction.

5.4 Tile width

	tiled		fused	
tile width	ms per step	TFLOP/s	ms per step	TFLOP/s
32	12.144	16.47	12.133	16.48
64	11.830	16.91	11.787	16.97
128	11.626	17.20	11.497	17.40
256	11.514	17.37	11.388	17.56

Table 5.5: Tile width sweep at 100,000 particles. Both variants time a whole step.

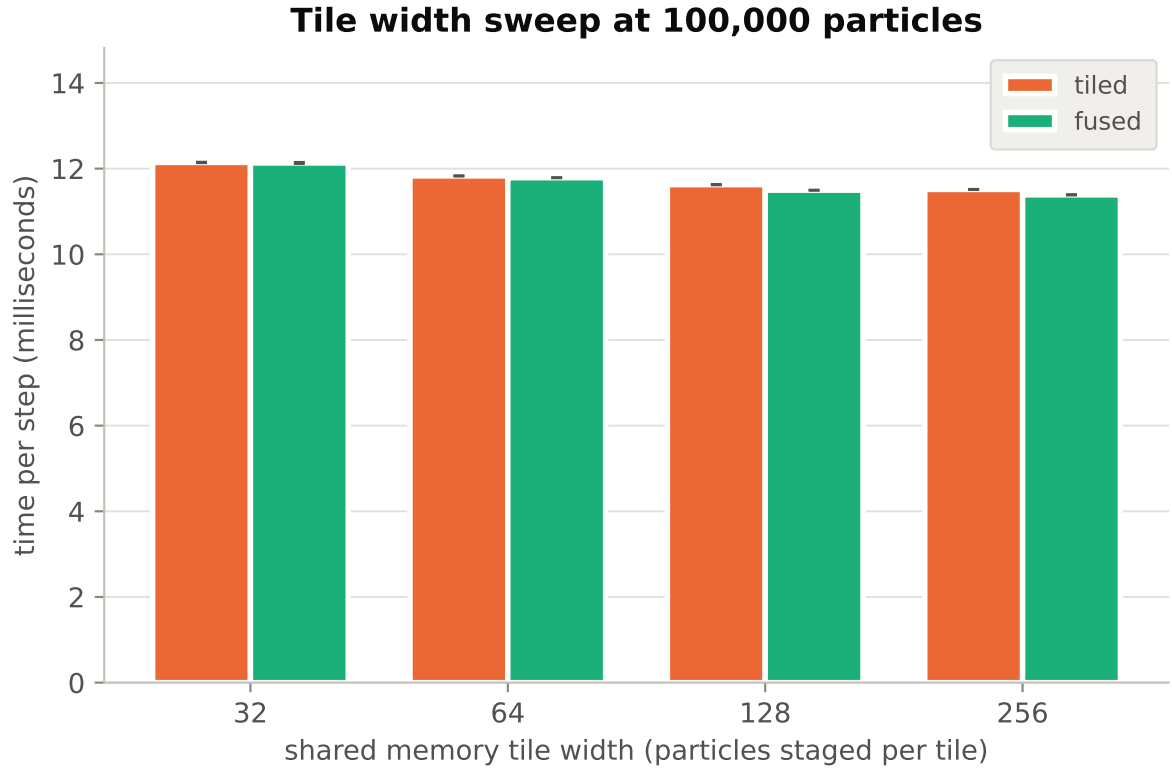


Figure 5.4: Step cost against shared memory tile width.

The default tile width is now 256, the fastest of the four measured. It runs at 11.514 milliseconds against 12.144 for a width of thirty two, the `v1.0.0` default, a difference of 5.2 percent. Taken round by round the ratio is 0.9486 with an interquartile range of 0.0020, well outside the session drift of 0.19 percent. Two barriers are paid per tile, so a narrow tile pays them more often per unit of arithmetic, which fits the direction of the result; the barrier cost itself was not measured separately.

particles	milliseconds per step	steps per second	million particles per second
10,000	0.0830	12,049	120.5
25,000	0.0969	10,324	258.1
50,000	0.1137	8,794	439.7
75,000	0.1537	6,507	488.0
100,000	0.1873	5,338	533.8

Table 5.6: Fluid step cost against particle count.

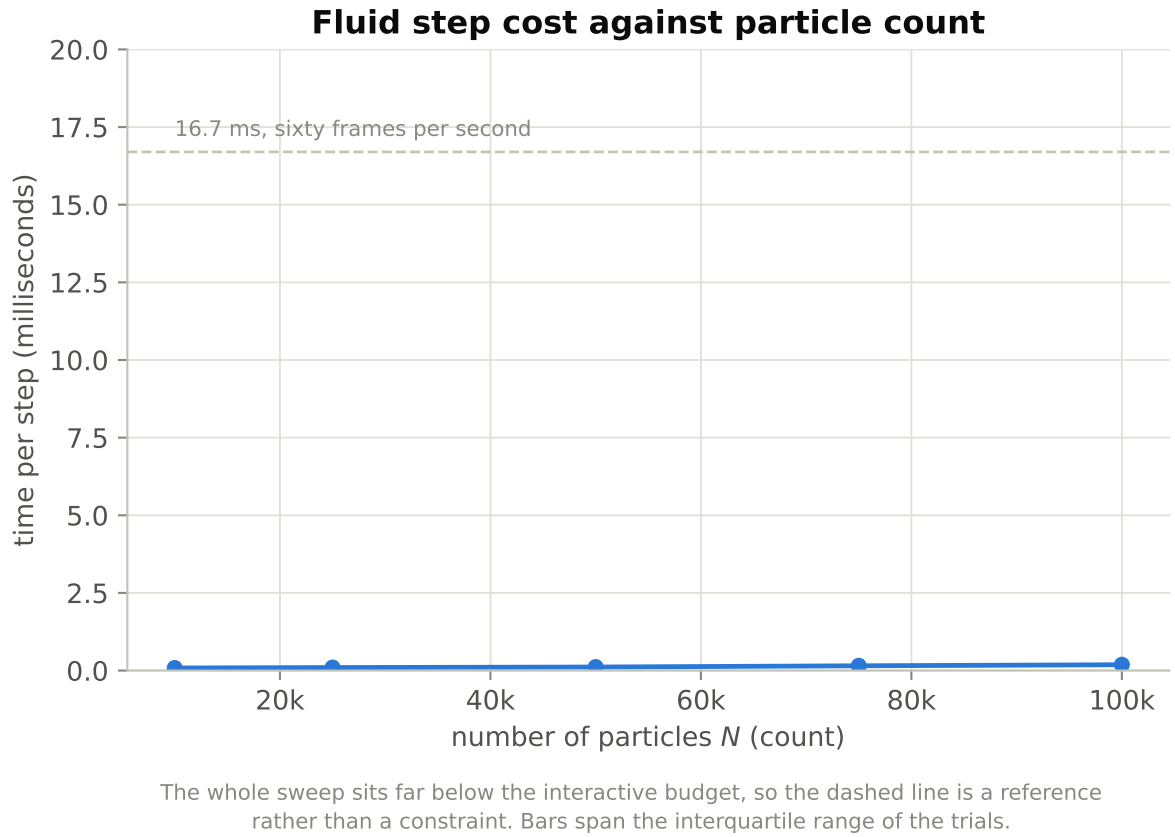


Figure 5.5: Fluid step cost against particle count.

5.5 The fluid solver

At fifty thousand particles, the size of the shipped dam break, a step costs 0.114 milliseconds, or 8,794 steps per second, which is 147 times inside a sixty frame per second budget. At 100,000 particles it costs 0.187 milliseconds, 5,338 steps per second. These results are not comparable with `v1.0.0`, because the particle mass changed and the fluid behaves differently. Nor can they be divided by another version's: the length of a trial depends on what a step costs, so a faster build times a later and cheaper stretch of the flow. Comparing versions takes a side by side run over the same steps.

The interquartile range of the fluid trials is 11.5 to 17.3 percent of the median, far above the session drift. That is the flow rather than noise: each engine stays alive across its trials, so a size advances through 4,000 timed steps, and the per round times trace a smooth curve as the fluid moves and its neighbour lists change.

Scaling is sublinear at the small end: 10 times the particles cost 2.3 times the time. The kernels of a step have a fixed cost whatever its size, which is a plausible reason, but the step has only been split pass by pass at one hundred thousand particles, where with graphs off 18.4 percent of it is time between kernels.

5.6 Physics gates

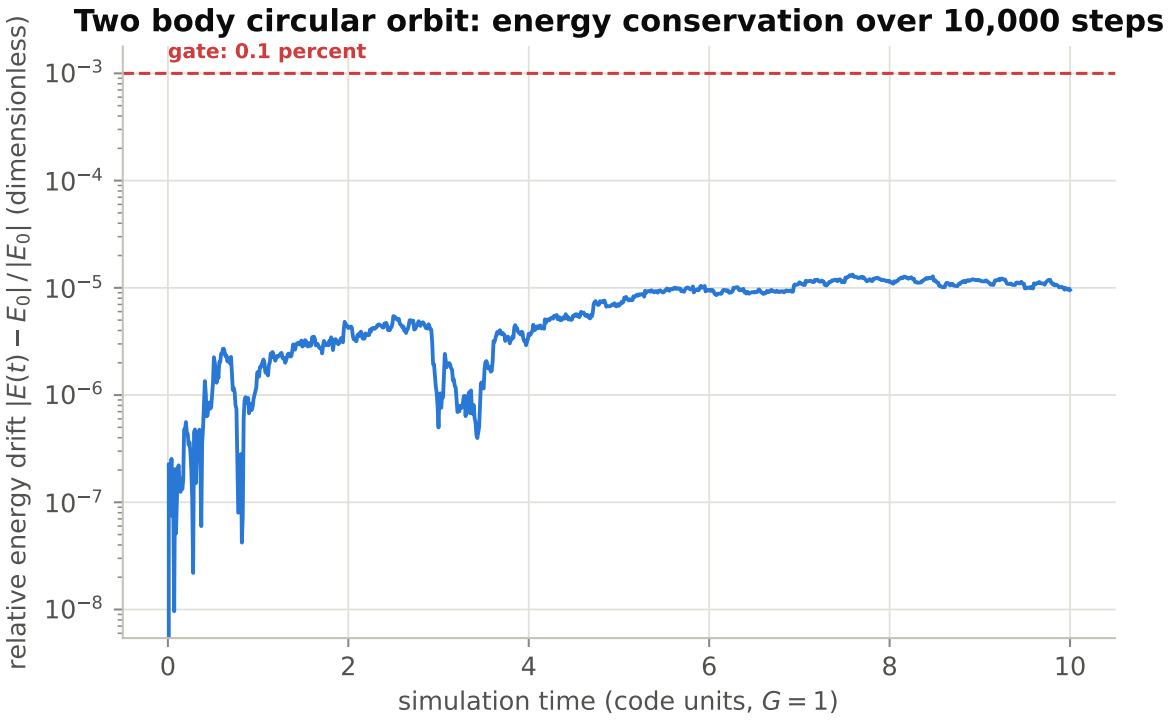
gate	measured	limit	verdict
Two body orbit, relative energy drift over 10,000 steps	1.33×10^{-5}	< 0.0010	**pass**
Two body orbit, departure from circular	9.35×10^{-6}	< 0.0010	**pass**
Velocity Verlet energy order, eccentric orbit, distance of slope from 2	0.0072	< 0.1500	**pass**
Velocity Verlet energy order, circular orbit, distance of slope from 4	0.0040	< 0.1500	**pass**
Plummer 10k, relative momentum drift over 1,000 steps	1.23×10^{-8}	< 0.0119	**pass**
Force against a double precision CPU reference, 100 particles	5.77×10^{-7}	$< 1.00 \times 10^{-5}$	**pass**
Pairwise force antisymmetry residual	1.53×10^{-7}	$< 1.00 \times 10^{-6}$	**pass**
Plummer radial density, chi squared p value	0.4416	> 0.0500	**pass**
SPH poly6 kernel normalisation error	6.48×10^{-8}	< 0.0010	**pass**
SPH spiky kernel normalisation error	4.39×10^{-8}	< 0.0010	**pass**
SPH interior density, mean error	2.81×10^{-4}	< 0.0100	**pass**
SPH interior density, RMS deviation	0.0149	< 0.0500	**pass**
SPH interior density, maximum deviation	0.0634	< 0.1000	**pass**
SPH neighbour search against a brute force sum	2.85×10^{-7}	$< 1.00 \times 10^{-4}$	**pass**

Table 5.7: Every physics gate, its measured value and its limit. All values come from the artifacts written by the validation suite, which links the library compiled with fast math off.

All 14 gates pass. The margins differ widely and not all of them mean the same thing: `docs/physics_validation.md` classifies each as a gate, a consistency check or a sanity check, and gives the source of its limit.

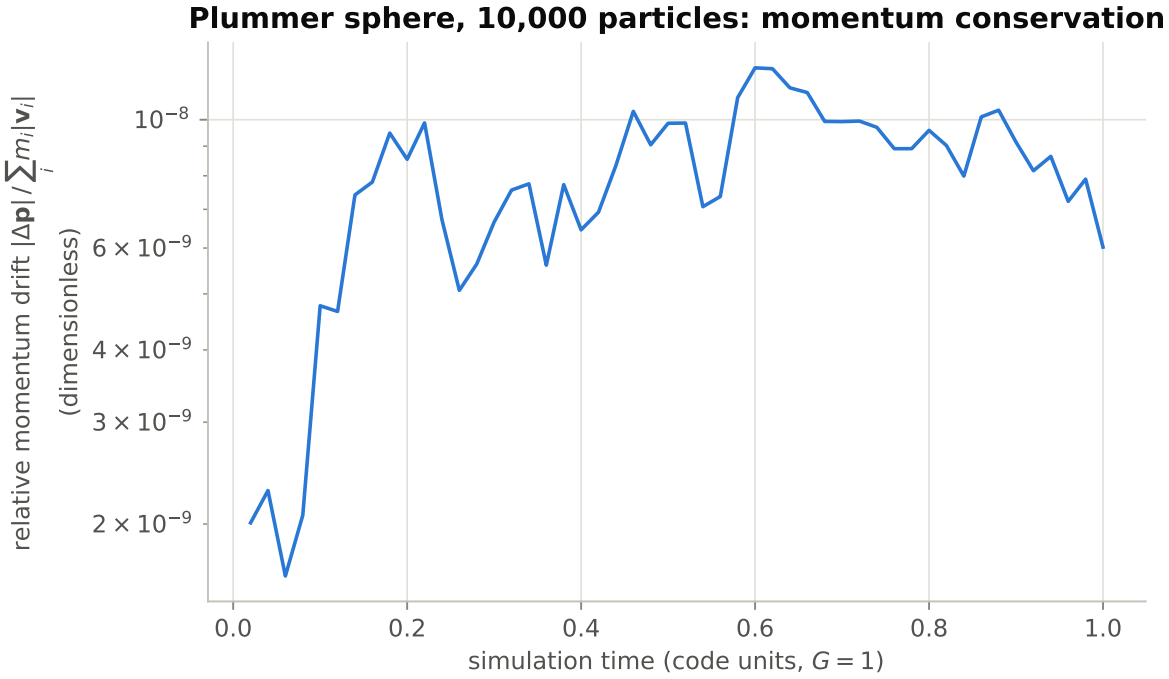
The two body energy error over ten thousand steps is 1.33×10^{-5} against a limit of one part in a thousand. That figure is dominated by single precision rounding: the same orbit in double precision has a truncation error of about 9.9×10^{-13}. So it is a regression test for the single precision path and says nothing about the integrator's order, and the shape of Figure 5.6 is the shape of rounding. `v1.0.0` read that shape as the signature of a symplectic integrator, which it cannot show.

Order is now tested in double precision. The slope of the energy error against the time step comes within 0.0072 of two on an orbit of eccentricity 0.5, which is the method's order, and within 0.0040 of four on an unsoftened circular orbit, where symmetry cancels the second order term. Both limits were committed before the code that measures them existed.



Time is in the code units of the run, where the gravitational constant, the total mass and the initial separation are all one, so one orbit is about 4.44 units. The error rises and then settles onto a bounded plateau instead of growing without limit, which is the property a symplectic integrator guarantees and a non symplectic one does not.

Figure 5.6: Relative energy error of the single precision two body orbit. At this step size it is single precision rounding, not integrator truncation.



Drift is measured against the total absolute momentum in the system, because the vector total is zero by construction and dividing by it would be dividing by noise. The rise is single precision rounding accumulating in the force sum.

Figure 5.7: Momentum conservation of a ten thousand particle Plummer sphere.

Momentum holds to 1.23×10^{-8} relative to the total absolute momentum in the system, six orders of magnitude inside the bound that single precision arithmetic can explain. A bound that far away only rules out gross breakage.

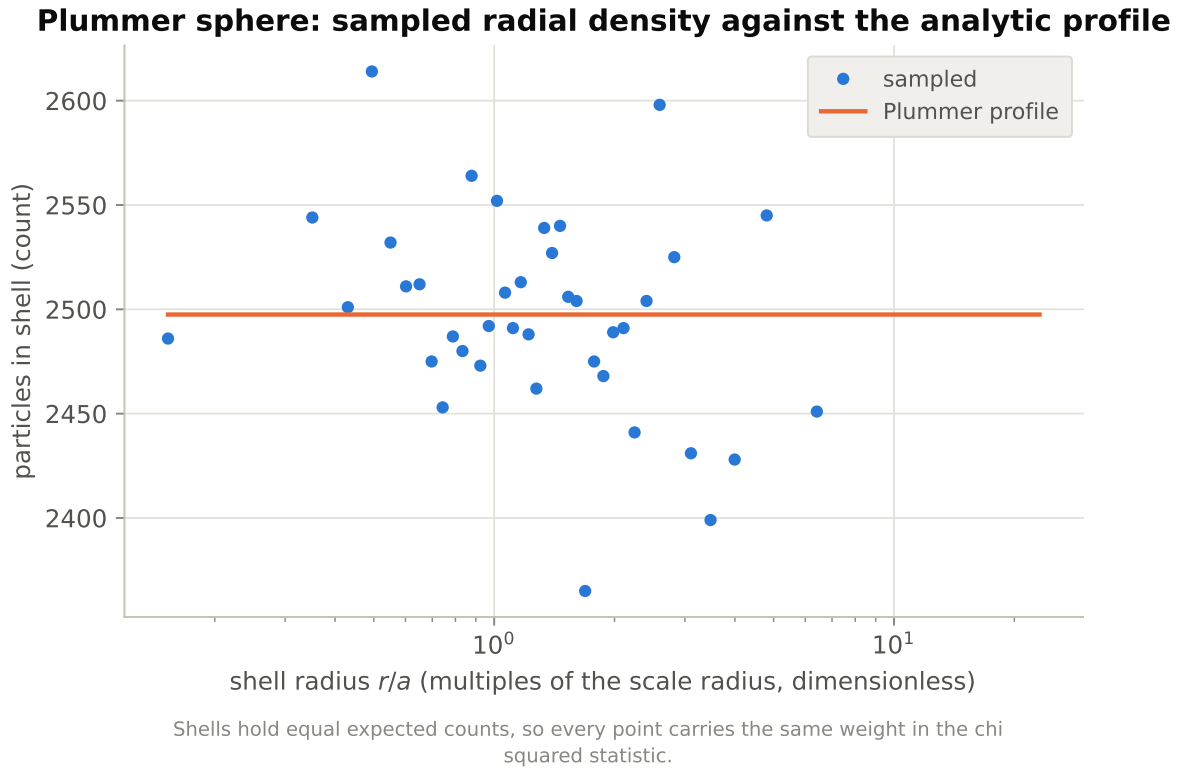


Figure 5.8: Sampled radial density of the Plummer sphere against the analytic profile.

The radial density fits the analytic profile at a chi squared p value of 0.4416, above the threshold of 0.05. With a fixed seed that value is a fixed seed regression and does not vary from run to run. The same test applied to a model whose scale radius is ten percent wrong returns a p value below 10^{-6}, which is what establishes that the test can reject at all.

For the fluid solver, the poly6 kernel integrates to one with an error of 6.48×10^{-8}, and the density field from the neighbour search agrees with a brute force $O(N^2)$ sum over the same positions to 2.85×10^{-7}.

The particle mass now comes from a kernel sum over the lattice actually laid out, so that the interior of the block sits at rest density. Against that calibration the interior mean density error is 0.03 percent. The calibration sets this value, so it shows the device agrees with the host calculation and is not evidence of physical accuracy. The figures the calibration does not set are the RMS deviation of 1.49 percent, which lattice jitter drives, and the maximum deviation of 6.34 percent. Chapter 8 records what `v1.0.0` reported here and why it was wrong.

Chapter 6

Discussion

6.1 Fusion and what the bytes could not say

The expectation of ten to twenty percent was not arbitrary; it is the sort of figure a memory bound kernel might give. What it missed is arithmetic intensity. The all pairs sum performs $O(N^2)$ arithmetic on $O(N)$ data, so the ratio of operations to bytes grows without limit as the problem grows. At 200,000 particles a force evaluation is 4×10^{10} interactions over a working set of a few megabytes, and the counters put device memory throughput at 0.32 percent of peak. Most of the step is arithmetic, so removing memory traffic from it was never going to give ten percent.

The general lesson is that an optimisation should be sized against the resource it relieves before it is implemented. But `v1.0.0` drew that lesson too far, by treating the result as a ceiling on what fusion could do. Pricing the removed bytes at streaming bandwidth bounds only the cost of those bytes at that bandwidth. The fused kernel beats that figure by several times at every size where the difference is resolved, and at 200,000 particles the comparison is not resolved at all. The arithmetic was right about the expectation and wrong as a prediction of the result, and only the paired measurement could tell the two apart.

That paired measurement also corrected a method. `v1.0.0` measured each variant in its own block, so its comparison of fused and tiled was confounded with whichever way the session was drifting. Interleaving the variants round by round is what made a difference of one percent readable, and the same data is what exposed the bimodal tiled trials that a single median had hidden.

6.2 Tiling is not a bandwidth optimisation here

The second correction is subtler. Shared memory tiling is conventionally introduced as a way of reducing device memory traffic. On this problem that framing is wrong, and the counters say so: device memory throughput is under one percent of peak both before and after the change. Tiling still delivers a factor of 1.96, but the resource it relieves is the first level cache and the load store pipeline, not the memory bus. Global load sectors fall by a factor of 63.9, and the naive kernel's first level cache throughput of 88.0 percent of peak identifies it as the true bottleneck.

This matters practically. Attacking the naive kernel's device memory traffic would not have addressed the unit the counters name as limiting it.

6.3 Where the first design and the measurement disagreed

Three points in the original design were changed or contradicted by measurement, and each is recorded rather than quietly absorbed.

The first design asked for GCC 16.1 as the host compiler, which is not achievable with CUDA 13.3, because it refuses any GCC newer than 15. GCC 15.2 supplies the C++23 host standard that floor was for.

A tile width of thirty two costs 5.2 percent against a width of 256 on this card. `v1.0.0` kept thirty two and reported the sweep. Under the current protocol the difference is well outside the drift, so the default is now 256, and the ladder in this report was measured there.

The first design's runtime estimates were too long. The validation suite was expected to be dominated by the ten thousand step orbit, but with two particles that orbit is cheap.

6.4 Limitations

The all pairs solver is $O(N^2)$ and always will be. At two hundred thousand particles a step costs 43.21 milliseconds; at two million it would cost a hundred times that. The range where this approach is the right one is bounded, and beyond it a tree code is not an optimisation but a different algorithm.

The softening in the shipped runs does not make them collisionless. At $\varepsilon = 0.01$ in a ten thousand particle Plummer sphere of scale radius one, the softening is about 7.5 times below the mean separation, so close encounters still matter and the run is mildly collisional. Here softening keeps the integrator stable; it is not a claim about the regime.

The fluid solver is weakly compressible, so it enforces incompressibility only through a stiff equation of state. That bounds the time step: the sound speed implied by the stiffness constant sets a limit far tighter than the fluid velocity would. The solver computes that limit and refuses a time step above it, and because the smoothing radius shrinks with particle count, the shipped time step of 10^{-4} seconds is refused above about one hundred and ninety thousand particles unless the check is overridden. The step rates in Section 5.5 are steps per second and not seconds of simulated time per second.

The fluid defaults are not those of water. The default viscosity of 0.2 Pa s is a kinematic viscosity of 2×10^{-4} square metres per second, about two hundred times that of water, and the wall damping of 0.4 keeps 16 percent of the normal kinetic energy at each impact. They are documented rather than changed here, because changing them would change every fluid number. The start is also not quiet: maximum speed after 100 steps is about six times what free fall alone gives, from a density scatter of about 1.5 percent acting through the stiff equation of state.

Single precision is a deliberate choice and it has a cost. The energy gate shows it: its value is rounding, not truncation. A long term integration of a planetary system would need double precision positions.

The benchmark measures the fluid from a block that starts at rest and collapses during measurement. A fully developed splash has a different spatial distribution and therefore a different neighbour list length, so the fluid timings should be read as representative of the first few thousand steps rather than of every phase of the flow.

Chapter 7

Conclusion

Direct N-body gravity computes 926 billion pairwise interactions per second at 200,000 particles, which is 18.52 TFLOP/s at the conventional 20 operations per interaction, and the fluid solver runs 100,000 particles at 0.187 milliseconds per step and fifty thousand 147 times inside an interactive budget. All 14 physics gates pass, against a library compiled with fast math disabled so that no compiler flag can relax them.

The memory hierarchy ladder produced a clear answer and a clear correction. Shared memory tiling is worth a factor of 1.96, and the hardware counters tie that gain to a 63.9 times reduction in global load sectors relieving a load store pipeline near saturation, not to device memory bandwidth, which stayed below one percent of peak in every variant. Fusing the integrator into the force kernel was expected to be worth ten to twenty percent. It is measurably faster by about one percent over most of the range, 0.9905 as a ratio at one hundred and fifty thousand particles, and not resolved at 200,000, where the tiled trials are bimodal for a reason not yet understood. The traffic it removes is too small to explain the gain, and what does explain it has not been measured.

The parts of the design that held up did so because they were chosen against a specific failure. The brute force comparison of the neighbour search was added because a dropped or double counted neighbour still looks like a fluid, and it held unchanged through a complete replacement of the search, agreeing to 2.85×10^{-7}. Double precision diagnostic accumulators were chosen because a single precision reduction would have measured itself rather than the integrator. Velocity Verlet's order is now confirmed by double precision gates; the single precision energy figure could not show it.

Other parts did not hold up, and Chapter 8 lists them, including a density gate that passed while measuring the wrong particles and a particle mass that started the fluid with a pressure pulse.

7.1 Further work

A Barnes Hut tree code is the obvious next step and the one that changes the reachable problem size rather than the constant factor. Its irregular traversal is a genuinely different GPU problem.

The fluid solver would benefit more from a position based or projection based incompressibility treatment than from any further kernel tuning, because the stiff equation of state, not the neighbour search, is what bounds the time step.

Three measurements are unfinished: the bimodal tiled trials at 200,000 particles, the source of the fusion gain, and the large tiling factor at 10,000. Stall profiling should separate them. The fused kernel also still writes an acceleration array nothing reads; dropping that write is a kernel change and will be measured against this baseline.

Chapter 8

Corrections to v1.0.0

`v1.0.0` made claims that were wrong. This chapter says what each one was, what was wrong with it, and what is true now. Values marked as `v1.0.0` values describe the past and are not regenerated.

8.1 Interior density

`v1.0.0` reported an interior density error of 4.20 percent against a 5 percent limit. That gate read densities in cell sorted order against positions in lattice order, so it measured mismatched particles. With the index fixed, the `v1.0.0` mass gives 7.61 percent, which fails that limit. The explanation `v1.0.0` gave, a finite smoothing radius on a jittered lattice, was also wrong: almost all of the residual was the mass, which did not match the lattice laid out.

That mass started the dam break interior near 76 kPa against a hydrostatic 3.9 kPa, so the `v1.0.0` fluid started with a pressure pulse. Maximum speed after 100 steps was 3.695 m/s and is now 0.625 m/s, still above the 0.098 m/s of free fall, so the start is better but not quiet. The error is now 0.03 percent by calibration, with the limits stated in Section 5.6.

8.2 Fusion

`v1.0.0` claimed a gain of 0.1 percent at its largest size, from a difference of 0.12 percent measured in separate blocks, with a session spread of about 3 percent. That difference could not be told from noise. It also described the saving as a round trip of the acceleration array, $24N$ bytes written and read, when the fused kernel still wrote that array, and called the resulting 0.019 percent an absolute ceiling, which the measurements in Section 5.2 now exceed. It said fusion halves the memory footprint of a step, which it does not: the fused path adds a second position buffer.

A first correction during this release said the saving was $12N$ bytes. That is also incomplete, because the split path reads the acceleration array in both integrator launches. Section 5.2 gives the traffic as the code has it now, and the paired ratios: fused is measurably faster up to one hundred and fifty thousand particles, and the difference is not resolved at 200,000, for a reason not yet understood.

8.3 Tile width

`v1.0.0` defaulted to a tile width of 32, and the default is now 256. Its tile sweep rows timed the force kernel alone for tiled, with particles that never moved, against whole steps for fused, so any comparison across variants in that table was void. Both variants now time a whole step.

8.4 Neighbour search

`v1.0.0` chose a spatial hash over a dense grid, because a dense grid needs storage proportional to the bounding volume and a dam break leaves most of that empty. That argument was wrong. The hash table had twice as many slots as particles, so it was larger than the grid it avoided, which at fifty thousand particles has 22,386 cells against 100,003 slots, by derivation from the domain and the smoothing radius. The search is now that grid, which also removed the per candidate key comparison that hashing had made necessary.

8.5 Provenance

Both commit hashes recorded in the `v1.0.0` results do not exist in the published history, because the history was rewritten before release. Every row now carries a hash that resolves, here `6823f76a56af`, and a dirty flag, and `scripts/check_provenance.py` refuses a row that fails either.

8.6 Energy gate

The two body energy figure is dominated by single precision rounding, 9.9×10^{-13} of truncation against 1.33×10^{-5} measured, so it says nothing about the integrator's order. `v1.0.0` presented it as evidence about the integrator. Order is now tested in double precision.

8.7 Numbers and prose that had drifted

The Markdown documents quoted values that disagreed with the recorded results, among them a streaming bandwidth of 545.8 GB/s and a throughput of 17.47 TFLOP/s, both `v1.0.0` values that no longer match any file. They now carry marks that a script checks against the recorded results. In the reports, `v1.0.0` said the tiling advantage grows with particle count, which its own table contradicted; said the debug report has eleven entries when it had fourteen; and called a tile width of 32 the slowest of the four measured when its sweep had 64 slower. The benchmark protocol also changed, so timings here are not directly comparable with `v1.0.0`.

Bibliography

[1] CUDA core compute libraries: CUB device wide primitives, 2026.

[2] Sverre J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.

[3] Sverre J. Aarseth, Michel Hénon, and Roland Wielen. A comparison of numerical methods for the study of star cluster dynamics. *Astronomy and Astrophysics*, 37:183 to 187, 1974.

[4] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration: Structure Preserving Algorithms for Ordinary Differential Equations*. Springer, 2 edition, 2006.

[5] Joseph J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30:543 to 574, 1992.

[6] Matthias Müller, David Charypar, and Markus Gross. Particle-based fluid simulation for interactive applications. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, page 154 to 159, 2003.

[7] NVIDIA Corporation. *CUDA C++ Programming Guide, version 13.3*, 2026.

[8] Lars Nyland, Mark Harris, and Jan Prins. Fast n-body simulation with CUDA. In Hubert Nguyen, editor, *GPU Gems 3*, chapter 31, page 677 to 695. Addison Wesley, 2007.

[9] Henry C. Plummer. On the problem of distribution in globular star clusters. *Monthly Notices of the Royal Astronomical Society*, 71:460 to 470, 1911.

[10] Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomeranets, and Markus Gross. Optimized spatial hashing for collision detection of deformable objects. In *Proceedings of Vision, Modeling and Visualization*, page 47 to 54, 2003.

[11] Loup Verlet. Computer experiments on classical fluids: Thermodynamical properties of lennard-jones molecules. *Physical Review*, 159(1):98 to 103, 1967.